

Tema 7. Métodos de stream

Autor: José C. Riquelme

Introducción

El avance que representa Java 8 con respecto a versiones anteriores es muy significativo y nos va a obligar a modificar la perspectiva de cómo enseñar a programar a nuestros estudiantes. Estos apuntes intentan ser un resumen de las principales características que hemos encontrado en un apresurado estudio de las nuevas funcionalidades del tipo Stream. Todo el código aquí presentado ha sido implementado usando Eclipse Versión: Luna (4.4) Build id: I20140501-0200.

Para ejemplificar los casos de uso partimos de un tipo Vuelo con la siguiente interfaz:

```
public interface Vuelo extends Comparable<Vuelo>{
    String getCodigo() ;
    String getDestino() ;
    Fecha getFecha();
    Integer getNumPasajeros();
    Integer getNumPlazas() ;
    Duracion getDuracion();
    Double getPrecio() ;
    Double getOcupacion(); //opcional
    Double getRecaudacion(); //opcional
    void setPrecio(Double p);
    void setDuracion(Duracion d);
    void setNumPasajeros(Integer np) ;
}
```

El tipo Duracion implementa el tiempo de un Vuelo y tiene la funcionalidad siguiente

```
public interface Duracion extends Comparable<Duracion>{
    Integer getMinutos();
    Integer getHoras();
    Duracion suma(Duracion d);
}
```

El objetivo es implementar un tipo Aeropuerto que manipula como atributo una colección de objetos de tipo Vuelo que hemos denominado `vuelos` y requiere una serie de funcionalidades que se irán presentado en cada uno de los siguientes secciones en que se ha dividido el texto para facilitar su comprensión. Las interfaces funcionales como Predicate, Function, Supplier etc. son explicadas en el tema anterior, aunque realmente su uso es relativamente fácil y no necesitan conocerse en profundidad para poder profundizar en la forma de programar que explicaremos en este tema.

Nos centramos en este documento en la estructura Stream que es un iterable virtual sobre objetos. El método `stream()` aplicado a una Collection permite obtener un Stream a partir de los objetos de la Collection. Por eso siempre el código de las siguientes secciones comienza con una sentencia `vuelos.stream()`. A continuación se describen las principales funcionalidades del tipo Stream.

1. Primeros Métodos

En esta primera sección se explican métodos que no conllevan el argumento de interfaces funcionales y que ofrecen estadísticas simples o constructores de stream a partir de otros.

1.1 Estadísticas simples

1.1.1 count()

Devuelve el número de elementos del stream que invoca.

1.1.2 sum() y average()

Sólo para streams numéricos (IntStream, LongStream, DoubleStream), devuelve la suma y la media de los valores del stream que invoca. El método sum devuelve la suma en el mismo tipo del stream. El método average devuelve siempre un OptionalDouble un tipo de dato que tiene Java 8 para indicar que puede tener un valor Double si todo ha sido correcto, o un valor nulo si por ejemplo el stream estaba vacío. El tipo OptionalDouble se manipula con los métodos isPresent() que nos indica si el valor es válido y getAsDouble() para obtener el valor double en el caso de que sea válido.

1.2 Métodos manipuladores

1.2.1 limit(long n)

Devuelve un nuevo stream con los primeros n elementos del stream original.

1.2.2 skip(long n)

Devuelve un nuevo stream en el que se han eliminado los n primeros elementos del stream que invoca.

1.2.3 distinct()

Devuelve un nuevo stream con sólo los elementos que son diferentes del stream original.

1.2.4 of(T ... valores)

Devuelve un stream formado con los argumentos pasados como parámetros al método.

1.2.5 concat(Stream s1, Stream s2)

Devuelve un nuevo stream resultado de concatenar los dos streams que son parámetros del método.

2. filter(Predicate)

Sirve para devolver otro Stream con sólo aquellos elementos del que invoca que cumplen el predicado. Se suele componer con métodos como count para calcular el número de elementos

de una colección que cumplen una determinada condición. Por ejemplo, cuántos vuelos hay en una fecha determinada:

```
public Long getNumVuelosDia(LocalDate f) {  
    return vuelos.stream().  
        filter(x->x.getFecha().equals(f)).  
        count();  
}
```

O cuántos vuelos completos hay:

```
public Long getNumVuelosCompletos() {  
    return vuelos.stream().  
        filter(x->x.getNumPasajeros().equals(x.getNumPlazas())).  
        count();  
}
```

Si un objeto de tipo Predicate se va a usar a menudo, debe definirse previamente y después invocarse. Por ejemplo, la condición de si un vuelo es de una determinada fecha, se definiría:

```
Predicate<Vuelo> fechaIgual(LocalDate f){  
    return x -> x.getFecha().equals(f);  
}
```

Y después se invocaría:

```
public Long getNumVuelosDia(LocalDate f) {  
    return vuelos.stream().  
        filter(fechaIgual(f)).  
        count();  
}
```

También se puede combinar con métodos como `findFirst` que devuelve el primer objeto de un stream o `limit(long)` que devuelve un stream limitado al número de elementos que se recibe como argumento.

3. allMatch(Predicate)

Nos devuelve un valor de cierto si todos los elementos de una colección cumplen una condición. Por ejemplo si nos preguntamos si todos los vuelos de una fecha están completos:

```
public Boolean todosCompletos(LocalDate f){  
    return vuelos.stream().  
        filter(fechaIgual(f)).  
        allMatch(x->x.getNumPasajeros().equals(x.getNumPlazas()));  
}
```

En este ejemplo el Predicate del filter se podía haber añadido al Predicate del método `allMatch` con un método `and` del tipo Predicate.

```
Predicate<Vuelo> vueloCompleto = x->  
    x.getNumPasajeros().equals(x.getNumPlazas());  
  
public Boolean todosCompletos2(LocalDate f){
```

```
        return vuelos.stream().
            allMatch(fechaIgual(f).and(vueloCompleto()));
    }
```

Hay que tener en cuenta que este segundo método todosCompletos2 no funciona igual al primero. La diferencia está en que todosCompletos primero realiza una operación de filtro, si el resultado es un stream vacío, es decir, ningún vuelo cumple la condición de fecha, entonces allMatch devolverá true. En el segundo método si ningún vuelo cumple el filtro de fecha el resultado será false.

4. anyMatch(Predicate)

Nos devuelve un valor de cierto si algún elemento de una colección cumple una condición. Por ejemplo si nos preguntamos si hay algún vuelo a un destino en una fecha concreta:

```
public Boolean hayVueloDestinoFecha(LocalDate f, String d){
    return vuelos.stream().
        anyMatch(x->x.getFecha().equals(f) &&
            x.getDestino().equals(d));
}
```

O si hay algún vuelo no completo un determinado día, definiendo previamente los objetos Predicate fechaIgual y vueloCompleto:

```
public Boolean hayVueloFechaNoCompleto(LocalDate f){
    return vuelos.stream().
        anyMatch(fechaIgual(f).and(vueloCompleto().negate()));
}
```

5. min(Comparator) y max(Comparator)

Devuelven el mínimo o máximo elemento de una colección según el orden establecido por un Comparator. Para definir el Comparator de un tipo por alguno de sus atributos es muy útil el método comparing de la clase Comparator. Por ejemplo, el vuelo más barato a un determinado destino se calcula comparando dos objetos de tipo Vuelo por su precio usando la Function Vuelo::getPrecio:

```
public Vuelo getVueloMasBaratoDestino(String d){
    return vuelos.stream().
        filter(x->x.getDestino().equals(d)).
        min(Comparator.comparing(Vuelo::getPrecio)).
        get();
}
```

La clase Comparator permite especificar el tipo que se está comparando. Así en el ejemplo anterior se debería hacer uso del método comparingDouble ya que ese es el tipo devuelto por getPrecio. Igualmente existen los métodos comparingInt o comparingLong. También se puede usar el orden natural de Vuelo (dado por su compareTo) mediante la invocación:

```
min(Comparator.naturalOrder());
```

Hay que tener en cuenta que el método `min` devuelve lo que en Java 8 se denomina un tipo opcional, ya que podría no existir, si por ejemplo, estuviéramos calculando el mínimo de una colección vacía que podría suceder si en el ejemplo anterior no hubiera ningún vuelo al destino que se pasa como argumento. Por eso se invoca después el método `get`, que devolvería el objeto menor si existe y sino lanza la excepción `NoSuchElementException`. Esta posibilidad puede ser controlada con otros métodos disponibles después de invocar a `min`. Como por ejemplo el método `ifPresent` que nos devuelve `true` si efectivamente existe mínimo o `orElse` que permite crear y devolver un objeto si el mínimo no existe.

Un ejemplo con el máximo sería si quisiéramos devolver el Vuelo de mayor ocupación un determinado día, implementaríamos el método:

```
public Vuelo getVueloMayorOcupacion(LocalDate f) {  
    return vuelos.stream().  
        filter(x->x.getFecha().equals(f)).  
        max(Comparator.comparingDouble(  
            x->x.getNumPasajeros()/x.getNumPlazas())).  
        get();  
}
```

En el caso anterior la `FunctionDouble` se ha definido como argumento del método `comparingDouble`. Otra solución distinta hubiera sido si `Vuelo` tuviera un método que devolviera su ocupación. Entonces el código quedaría:

```
public Vuelo getVueloMayorOcupacion(LocalDate f) {  
    return vuelos.stream().  
        filter(x->x.getFecha().equals(f)).  
        max(Comparator.comparingDouble(Vuelo::getOcupacion))  
        get();  
}
```

Si el tipo `Vuelo` no tuviera esa funcionalidad se podría definir una `Function` para ello:

```
Function<Vuelo, Double> getOcupacion = x->  
    1.*x.getNumPasajeros()/x.getNumPlazas();
```

O una `ToDoubleFunction`:

```
ToDoubleFunction<Vuelo> getOcupacion = x->  
    1.*x.getNumPasajeros()/x.getNumPlazas();
```

Quedando entonces el método:

```
public Vuelo getVueloMayorOcupacion(LocalDate f) {  
    return vuelos.stream().  
        filter(x->x.getFecha().equals(f)).  
        max(Comparator.comparingDouble(getOcupacion))  
        get();  
} //getOcupacion se refiere a la definida como ToDoubleFunction  
// para usar getOcupacion como Function debería invocarse desde comparing
```

6. map(Function)

Este método es realmente una familia de métodos con sus adaptaciones `mapToInt(ToIntFunction)`, `mapToLong(ToLongFunction)` y `mapToDouble(ToDoubleFunction)` devolviendo un Stream de objetos de otro tipo obtenidos a partir del tipo base aplicando una Function. Si el tipo devuelto es Integer, Long o Double existen los métodos específicos con las correspondientes funciones como argumentos. Así si en el ejemplo anterior no nos interesara cuál es el Vuelo más completo sino cuál es la ocupación máxima, se podría implementar el método:

```
public Double getMayorOcupacion(LocalDate f) {  
    return vuelos.stream().  
        filter(x->x.getFecha().equals(f)).  
        mapToDouble(x->x.getNumPasajeros()/x.getNumPlazas()).  
        max().  
        getAsDouble();  
}
```

O usando la ToDoubleFunction anterior:

```
public Double getMayorOcupacion(LocalDate f) {  
    return vuelos.stream().  
        filter(x->x.getFecha().equals(f)).  
        mapToDouble(getOcupacion()).  
        max().getAsDouble();  
}
```

Los métodos map también nos permiten facilitar una serie de operaciones sobre los valores devueltos. Por ejemplo si quisiéramos calcular la recaudación total de los vuelos a un determinado destino escribiríamos:

```
public Double sumaRecaudacionDestino(String d) {  
    return vuelos.stream().  
        filter(x->x.getDestino().equals(d)).  
        mapToDouble(x->x.getNumPasajeros()*x.getPrecio()).  
        sum();  
}
```

Igual que se calcula la suma se podría devolver la media con el método average. Por ejemplo definiendo una ToDoubleFunction:

```
ToDoubleFunction<Vuelo> getRecaudacion = x->  
    x.getNumPasajeros()/x.getPrecio();
```

La recaudación media de un determinado día quedaría:

```
public Double mediaRecaudacionFecha(LocalDate f) {  
    return vuelos.stream().  
        filter(fechaIgual(f)).  
        mapToDouble(getRecaudacion).  
        average().  
        getAsDouble;  
} // average devuelve un OptionalDouble por si no hubiera elementos
```

7. reduce(T, BiFunction)

El método reduce produce una “reducción” de un stream, mediante la operación dada por la Bifunction usando como elemento neutro el primer argumento. Las operaciones sum o average vistas anteriormente son casos particulares de reduce para tipos numéricos. Para otros tipos creados por el programador como Duracion, con el método reduce se puede sumar las duraciones de los vuelos de un determinado día:

El Tipo Duracion debe definir un método suma capaz de devolver la suma de dos objetos.

```
public Duracion suma(Duracion d) {  
    Integer min = getMinutos() + d.getMinutos();  
    Integer hor = getHoras() + d.getHoras();  
    return new DuracionImpl(hor+min/60,min%60);  
}
```

El método reduce se invoca con dos argumentos: el primero es el elemento neutro que inicializa el acumulador y el segundo la operación que acumula:

```
public Duracion getDuracionVuelosFecha(LocalDate f){  
    return vuelos.stream().  
        filter(x->x.getFecha().equals(f)).  
        map(Vuelo::getDuracion).  
        reduce(new DuracionImpl(0,0), Duracion::suma);  
}
```

Otra forma de hacerlo es definir la BiFunction en el código. En este caso como los tres tipos que intervienen son de tipo Duracion es un caso particular de BinaryOperator que se podría definir:

```
BinaryOperator<Duracion> sumadur = (x,y)-> x.suma(y);
```

Incluso si el tipo Duracion no implementara la funcionalidad suma se podría hacer en el código:

```
BinaryOperator<Duracion> sumadur = (x,y)-> {  
    Integer min = x.getMinutos() + y.getMinutos();  
    Integer hor = x.getHoras() + y.getHoras();  
    return new DuracionImpl(hor+min/60,min%60);  
};
```

De esta manera la llamada al método reduce quedaría:

```
reduce(new DuracionImpl(0,0), sumadur);
```

8. forEach(Consumer)

El método forEach sirve para llevar a cabo una acción definida por el objeto Consumer que se pasa como argumento sobre todos los elementos del stream. Por ejemplo para cambiar las duraciones de los vuelos a un determinado destino en un número determinado de minutos, se construiría un método como el siguiente:

```

public void incrementaDuracionDestino(String d, Integer min){
    vuelos.stream().
        filter(x->x.getDestino().equals(d)).
        forEach(x->x.setDuracion(x.getDuracion().suma(new
                                                    DuracionImpl(0,min))));
}

```

O para incrementar los precios de los vuelos un 10% a partir de un día determinado:

```

Consumer<Vuelo> incrementaPrecio10p = x->x.setPrecio(x.getPrecio()*1.1);

public void incrementaPrecios10pAPartirFecha(Fecha f) {
    vuelos.stream().
        filter(x->x.getFecha().compareTo(f)>0).
        forEach(incrementaPrecio10p);
}

```

También serviría para hacer un método estático que escriba en un fichero de texto los elementos de un stream uno por línea:

```

public static <T> void escribeFichero(Stream<T> it, String filename){
    File file = new File(filename);
    try {
        PrintWriter ps = new PrintWriter(file);
        it.forEach(x->ps.println(x));
        ps.close();
    } catch (FileNotFoundException e) {
        System.out.println("Fichero no encontrado "+filename);
    }
}

```

9. sorted(Comparator)

El método sorted devuelve el stream ordenado por el Comparator que se pasa como argumento. Sin el argumento el orden sería el natural. Por ejemplo, reutilizando el método estático anterior para escribir un Stream en un fichero, podríamos implementar el siguiente método que escribe un Stream ordenado por Fecha y Duracion.

```

public void escribeVuelosOrdenadosFechaDuracion (String fileName){
    Util.escribeFichero(vuelos.stream().
        sorted(Comparator.comparing(Vuelo::getFecha).
            thenComparing(Vuelo::getDuracion)), fileName);
}

```

10. collect(Collector)

La funcionalidad collect es un potente mecanismo para reducir un stream en otra colección, estructura Map o dato. alguna de estas funcionalidades puede ser realizada por métodos vistos anteriormente, como reduce. Como las posibilidades son muy numerosas vamos a dividir las según el objetivo.

10.1 Reducción a List o Set. El método collect proporciona un mecanismo muy potente para transformar un stream en distinto tipo de colecciones de datos como List y Set. Por ejemplo, si se quisiera devolver una lista con las duraciones de los vuelos a un determinado destino, se escribiría:

```
public List<Duracion> getDuracionesDestino(String d){
    return vuelos.stream().
        filter(x->x.getDestino().equals(d)).
        map(x->x.getDuracion()).
        collect(Collectors.toList());
}
```

La clase Collectors proporciona dos métodos toList y toSet de forma que el Stream de objetos Duracion devuelto por la operación map puede ser convertida en una lista o un conjunto respectivamente. Por ejemplo, el conjunto de los destinos posibles desde un aeropuerto viene dado por el método:

```
public Set<String> getDestinos(){
    return vuelos.stream().
        map(x->x.getDestino()).
        collect(Collectors.toSet());
}
```

Esta unión entre map y collect se puede hacer en una sola operación con el método mapping de Collectors:

```
public Set<String> getDestinos(){
    return vuelos.stream().
        collect(Collectors.mapping(x->x.getDestino(),
                                   Collectors.toSet()));
}
```

Los métodos toList y toSet son casos particulares del método toCollection que recibe un argumento de tipo Supplier con, por ejemplo, una invocación al constructor del tipo. De esta manera podemos devolver un SortedSet con los destinos de un determinado día:

```
public SortedSet<String> getDestinosEnFecha(Fecha f){
    return vuelos.stream().
        filter(x->x.getFecha().equals(f)).
        map(x->x.getDestino()).
        collect(Collectors.toCollection(TreeSet::new));
}
```

También se pueden transformar los objetos antes de su recolección. Por ejemplo, si quisiéramos obtener una lista con las duraciones de los vuelos a un determinado destino incrementadas en m minutos, se implementaría el siguiente método:

```
public List<Duracion> getDuracionesIncrementadasDestino(String d, Integer m){
    return vuelos.stream().
        filter(x->x.getDestino().equals(d)).
        map(x->x.getDuracion().suma(new DuracionImpl(0,m))).
        collect(Collectors.toList());
}
```

10.2 Estadísticas. Si solo quisiéramos contar el número de elementos del stream se puede usar también `Collectors.counting()`. Igualmente si el stream es de datos numéricos se pueden usar los colectores `Collectors.summingDouble` (`summingInt` o `summingLong`) para devolver la suma del stream en el mismo tipo y `Collectors.averagingDouble` (`averagingInt` o `averagingLong`) que devuelven la media aritmética siempre en tipo `Double`.

También existe el tipo `DoubleSummaryStatistics` a partir del cual es posible obtener los valores de la suma, máximo, mínimo y media de una serie de valores numéricos. Un objeto de este tipo es devuelto por `Collectors.summarizingDouble` que recibe una `ToDoubleFunction` (también existen igualmente `summarizingInt` y `summarizingLong`). Por ejemplo, para obtener la media de los precios de los vuelos a un determinado destino pondríamos:

```
public Double getMediaPreciosDestino(String d){
    return vuelos.stream().
        filter(x->x.getDestino().equals(d)).
        collect(Collectors.summarizingDouble(Vuelo::getPrecio)).
        getAverage();
}
```

Teniendo en cuenta que la varianza de una muestra es la media de los valores al cuadrado menos la media aritmética al cuadrado, se podría calcular la varianza de todos los precios del aeropuerto con el siguiente método:

```
public Double getVarianzaPrecios(){
    Double medcuad = vuelos.stream().
        collect(Collectors.summarizingDouble(
            x->x.getPrecio()*x.getPrecio())).
        getAverage();
    Double media = vuelos.stream().
        collect(Collectors.summarizingDouble(Vuelo::getPrecio)).
        getAverage();
    return medcuad-media*media;
}
```

Cuando no podemos emplear las operaciones aritméticas habituales de suma o media porque el tipo base no es un tipo numérico (`Double`, `Integer`, etc), se debe emplear la funcionalidad `reducing` de `Collectors`, para implementar nuevas operaciones sobre este tipo base. Por ejemplo si quisiéramos devolver la suma de las duraciones de todos los vuelos a un destino incrementadas en `m` minutos:

```
public Duracion incrementaySumaDuracionDestino(String d, Integer m){
    return vuelos.stream().
        filter(x->x.getDestino().equals(d)).
        map(x->x.getDuracion().suma(new DuracionImpl(0,m))).
        collect(Collectors.reducing(
            new DuracionImpl(0,0), Duracion::suma));
}
```

10.3 Reducción a Map. La clase `Collectors` proporciona tres posibles métodos `partitioningBy`, `toMap` y `groupingBy` para organizar la información de un stream en un `Map`. Las posibilidades son numerosas ya que los distintos métodos puede recibir diferentes argumentos.

10.3.1 groupingBy(Function) y partitioningBy(predicate). En este primer caso, la funcionalidad groupingBy recibe una Function, mientras que partitioningBy es un caso particular en el que la Function es sustituida por un Predicate. En este caso, el conjunto final es un List con los objetos del stream. Veamos distintos ejemplos de uso, si se quisiera construir un Map<Boolean, List<Vuelo>> para clasificar los vuelos del aeropuerto en completos o no, escribiríamos usando el Predicate declarado en el punto 2:

```
public Map<Boolean, List<Vuelo>> getMapVuelosCompleto(){
    return vuelos.stream().
        collect(Collectors.partitioningBy(vueloCompleto()));
}
```

Un poco más complejo es si quisiéramos obtener un Map<Boolean, List<Vuelo>> para clasificar los vuelos según hubieran pasado o no un determinado umbral de ocupación que se pasa como argumento:

```
public Map<Boolean, List<Vuelo>> getMapVuelosPorcentaje(Double p){
    return vuelos.stream().
        collect(Collectors.partitioningBy(
            x->x.getOcupacion().compareTo(p/100)>=0));
}
```

Para obtener un Map con más de dos valores en el conjunto inicial hay que recurrir a groupingBy. De esta manera para obtener un Map que organice los vuelos por fecha, escribiríamos:

```
public Map<LocalDate, List<Vuelo>> getMapVuelosXFecha(){
    return vuelos.stream().
        collect(Collectors.groupingBy(Vuelo::getFecha));
}
```

10.3.2 toMap(Function, Function). Este método devuelve un Map donde las claves vienen dadas por el primer argumento de tipo Function y los valores por el segundo. Por ejemplo, si queremos devolver un Map que haga corresponder a cada código de vuelo su destino.

```
public Map<String, String> getMapDestinosPorCodigo(){
    return vuelos.stream().
        collect(Collectors.toMap(Vuelo::getCodigo, Vuelo::getDestino));
}
```

El método toMap no permite claves repetidas, si las hubiera lanzaría la excepción IllegalStateException. Si el stream tiene objetos con clave repetida entonces debemos usar el toMap con tres argumentos explicado a continuación.

10.3.3 toMap(Function, Function, BinaryOperator). En este caso el tercer argumento es un BinaryOperator para que si hay dos registros con una misma clave los "reduzca" a uno. Por ejemplo si quisiéramos obtener un Map que relacionara los destinos con los códigos de los vuelos a ese destino, es claro existirían vuelos con el mismo destino, podemos realizar una concatenación de los String que representan los códigos de esos vuelos.

```
BinaryOperator<String> concatena=(s1, s2) -> s1 + "," + s2;
public Map<String,String> getMapCodigosPorDestino(){
    return vuelos.stream().
        collect(Collectors.toMap(Vuelo::getDestino, Vuelo::getCodigo,
                                concatena));
}
```

O el Map que relacione el total de pasajeros por fecha:

```
public Map<LocalDate,Integer> getMapNumPasajerosPorFecha(){
    return vuelos.stream().
        collect(Collectors.toMap(Vuelo::getFecha, Vuelo::getNumPasajeros,
                                (n1,n2) -> n1 + n2));
}
```

Hay dos métodos de la interfaz `BinaryOperator` para devolver el mínimo o el máximo de dos valores que se pueden usar para devolver un `Map` que relacione una clave con el mínimo o el máximo de una colección. Por ejemplo, un `Map` que relacione cada destino con el vuelo más barato a ese destino (Nótese el uso de la lambda expresión con la identidad):

```
public Map<String, Vuelo> getMapVueloMasBaratoPorDestino(){
    return vuelos.stream().
        collect(Collectors.toMap(Vuelo::getDestino, x->x,
            BinaryOperator.minBy(
                Comparator.comparing(Vuelo::getPrecio))));
}
```

10.3.4 groupingBy(Function, Collector). Por supuesto es posible que el tipo del conjunto final del Map no sea un List de objetos del stream sino que también podría ser un Set, pasando como argumentos a groupingBy además de la Function para el conjunto inicial un Collector para el conjunto final:

```
public Map<String, Set<Vuelo>> getMapSetVuelosXDestino(){
    return vuelos.stream().
        collect(Collectors.groupingBy(Vuelo::getDestino,
                                     Collectors.toSet()));
}
```

Si la información del conjunto final es una reducción u operación sobre los objetos del stream es necesario acudir a otros objetos de tipo `Collector` para pasarle como argumento a `groupBy`. De esta forma, usando los métodos de `Collectors` `counting`, `averaging` o `summing` se puede obtener una reducción del tipo del conjunto final del `Map`. Así para obtener el número de vuelos por destino el método sería:

```
public Map<String, Long> getMapNumVuelosXDestino(){
    return vuelos.stream().
        collect(Collectors.groupingBy(Vuelo::getDestino,
            Collectors.counting()));
}
```

Igualmente si se quiere devolver un Map con los precios medios por destino:

```
public Map<String, Double> getMapPrecioMedioXDestino(){
    return vuelos.stream().
        collect(Collectors.groupingBy(Vuelo::getDestino,
            Collectors.averagingDouble(Vuelo::getPrecio)));
}
```

O un Map con la recaudación por Fecha:

```
public Map<LocalDate, Double> getMapRecaudacionXFecha(){
    return vuelos.stream().
        collect(Collectors.groupingBy(
            Vuelo::getFecha,
            Collectors.summingDouble(
                (Vuelo x)->x.getNumPasajeros()*x.getPrecio())));
}
```

Que se podría escribir más sencillo usando la ToDoubleFunction getRecaudacion del punto 6:

```
public Map<LocalDate, Double> getMapRecaudacionXFecha(){
    return vuelos.stream().
        collect(Collectors.groupingBy(Vuelo::getFecha,
            Collectors.summingDouble(getRecaudacion())));
}
```

Otras veces se requiere que el tipo del conjunto final sea una transformación del tipo del stream. En ese caso el Collector que se pasa como segundo argumento de groupingBy es un Collectors.mapping que debe recibir una Function y otro Collector. Por ejemplo, para construir un Map con destinos en el conjunto inicial y una lista de los precios por destino en el conjunto final se escribiría:

```
public Map<String, List<Double>> getMapPreciosXDestino(){
    return vuelos.stream().
        collect(Collectors.groupingBy(Vuelo::getDestino,
            Collectors.mapping(Vuelo::getPrecio,
                Collectors.toList())));
}
```

10.3.5 groupingBy(Function, Supplier, Collector). Proporcionar un Supplier permite, entre otras posibilidades devolver SortedMap como resultado de groupingBy. De esta manera, para devolver un SortedMap con una lista de vuelos por Fecha, escribiríamos:

```
public SortedMap<Fecha, List<Vuelo>> getSortedMapVuelosXFecha(){
    return vuelos.stream().
        collect(Collectors.groupingBy(Vuelo::getFecha,
            TreeMap::new,
            Collectors.toList()));
}
```

10.4 Tratamientos más complejos con maps

Un ejercicio más complicado es invertir un Map. Es decir, si tenemos un Map<K,V> obtener un Map<V,List<K>> o Map<V,Set<K>> donde los elementos de los conjuntos inicial y final del Map original han intercambiado sus papeles en el Map inverso. Lógicamente como la aplicación original puede ser sobreyectiva es necesario asignar en el Map inverso a cada elemento del conjunto inicial varios del conjunto inicial. Por ejemplo, hemos hallado anteriormente un Map<String, Long> con el número de vuelos por destino. Si quisiéramos saber el destino con más vuelos, una solución sería invertir el Map anterior en un SortedMap<Long,List<String>> de manera que la clave mayor nos proporciona la lista de destinos con mayor número de vuelos. En Java 7 este método `invierteMap` se podía escribir como método estático así:

```
public static <T,K> SortedMap<T, List<K>> invierteMap(Map<K, T> m) {
    SortedMap<T, List<K>> res = new TreeMap<T,List<K>>();
    Set<K> sp = m.keySet();
    for(K elem: sp){
        T val =m.get(elem);
        if (res.containsKey(val)) {
            res.get(val).add(elem);
        }
        else {
            List<K> lista = new LinkedList<K>();
            lista.add(elem);
            res.put(val, lista);
        }
    }
    return res;
}
```

En Java 8 se usaría un código más compacto, dejando al lector si más fácil o no de entender. Se trata de convertir el Map original en un stream mediante el método `entrySet` que devuelve el conjunto de pares clave-valor. Una vez que tenemos el Map convertido en un stream de pares, se invoca a `collect` agrupando los pares por los valores (es decir, los valores pasan a ser las claves del nuevo Map) y haciendo que en el conjunto final estén List de las claves originales mediante el mapping correspondiente. El constructor `TreeMap` nos permite que el resultado sea un Sortedmap:

```
public static <K,T> SortedMap<K, List<T>> invierteMapAList(Map<T, K> m){
    return m.entrySet().stream().
        collect(Collectors.groupingBy(
            Map.Entry<T,K>::getValue,
            TreeMap::new,
            Collectors.mapping(Map.Entry<T,K>::getKey,
                               Collectors.toList())));
}
```

Cambiando el `toList()` final por un `toSet()` obtendríamos el método `invierteMapASet`. Una aplicación inmediata de esta estructura de código para el caso particular de obtener el número de vuelos relacionado con el conjunto de destinos es el siguiente método:

```
public SortedMap<Long, Set<String>> getMapDestinosXNumVuelos(){
    return getMapNumVuelosXDestino().
```

```

        entrySet().stream().
        Collect(Collectors.groupingBy(
            Map.Entry<String,Long>::getValue,
            TreeMap::new,
            Collectors.mapping(
                Map.Entry<String,Long>::getKey,
                Collectors.toSet())));
    }

```

De esta manera responder a la pregunta cuáles son los destinos con mayor número de vuelos se resuelve de la siguiente manera:

```

public Set<String> getDestinosMasVuelos(){
    SortedMap<Long,Set<String>> m = getMapDestinosXNumVuelos();
    return m.get(m.lastKey());
}

```

O empleando el método estático `invierteMapASet`:

```

public Set<String> getDestinosMasVuelos(){
    SortedMap<Long,Set<String>>m=Util.invierteMapASet(getMapNumVuelosXDestino());
    return m.get(m.lastKey());
}

```

Una versión más simple de este problema se puede resolver si sabemos que sólo hay un máximo y por tanto no es necesario devolver un Set con los posibles valores donde se da. Por ejemplo, si sólo hubiera un destino con el máximo de vuelos una solución sería partir del Map que asigna a cada destino su número de vuelos, convertir a stream su conjunto de pares valor, y después obtener el máximo mediante un comparador sólo con los valores y devolver el par que le corresponde al máximo mediante get y la clave de ese par mediante getKey:

```

public String getUnDestinoMasVuelos(){
    return getMapNumVuelosXDestino().
        entrySet().stream().
        max(Comparator.comparing(x->x.getValue())).
        get().
        getKey();
}

```

11. Ejercicios

Puede consultar más ejercicios resueltos en Java 8 en la sección de exámenes de la página del Curos de POO en Java del blog laprogramacionnoesunarte. También puede hallar más ejercicios resueltos en los enlaces:

http://laprogramacionnoesunarte.blogspot.com.es/p/blog-page_16.html

http://laprogramacionnoesunarte.blogspot.com.es/p/blog-page_5.html

11.1 Tipo Punto

Defina el tipo Punto con dos coordenadas reales, ambas consultables y modificables. Defina el TipoCuadrante con los valores PRIMER, SEGUNDO, TERCER, CUARTO y EJES y el TipoDiana con los valores INTERIOR, MEDIO y EXTERIOR. Un punto pertenecerá a algún cuadrante en función del signo de sus coordenadas según la representación cartesiana y si alguna de sus coordenadas es cero entonces estará sobre un eje.

Proporcione un método que calcule la distancia entre dos puntos y reutilícelo para otro método que devuelva la distancia al origen de coordenadas.

Un punto pertenecerá a alguno de los valores de TipoDiana según la distancia al origen sea respectivamente menor de 4, mayor de 4 y menor de 8 o mayor de 8. Proporcione al tipo Punto un método que devuelva su TipoDiana correspondiente.

Finalmente proporcione al tipo Punto un método tal que dados dos valores reales vx e vy desplace sus coordenadas en la dirección y sentido del vector (vx,vy). Defina ORIGEN como un atributo estático y final para la clase PuntoImpl.

Proporcione a PuntoImpl un constructor a partir de dos valores reales para sus coordenadas y a partir de un String con dos valores reales separados por una coma. Implemente los métodos toString, equals y compareTo. Dos puntos se ordenaran primero por su coordenada X y a igualdad de esta por su coordenada Y.

11.2 Tipo NubePuntos

Vamos a definir un tipo NubePuntos para guardar una colección de objetos de tipo Punto. La funcionalidad de este tipo será la siguiente:

1. Un método tal que dado un valor real nx devuelva el número de puntos que tienen abscisa mayor de nx.
2. Un método tal que dado un valor de TipoCuadrante devuelva el número de puntos en ese cuadrante, no olvide los puntos sobre los ejes.
3. Un método tal que dado un valor real d devuelva el número de puntos cuya distancia al origen es menor que d.
4. Devuelva un Punto que sea el centro de gravedad de la nube. Esto es, el punto cuyas coordenadas son la media de las coordenadas de los puntos.
5. Devuelva si existe algún punto sobre el eje de abscisas.
6. Dado un Punto p y un radio r devuelva si existe algún punto dentro del círculo de centro p y radio r.
7. Dado un valor real d, devuelve si es cierto que todos los puntos están a una distancia al origen menor que d.
8. Devuelve el Punto más lejano al origen de coordenadas.
9. Dado un Punto p, devuelva el punto de la nube más cercano.
10. Dado un valor real d, devuelva una NubePuntos con aquellos que estén a una distancia menor que d del origen.

11. Devuelva un Map que asocie a cada TipoCuadrante un SortedSet con los puntos de ese cuadrante ordenados por distancia al origen.
12. Devuelva un Map que asocie a cada TipoCuadrante el número de puntos que hay de ese tipo
13. Devuelva un SortedMap que asocie a cada TipoDiana un Set con los puntos que le corresponden.
14. Dados dos valores vx e vy desplace toda la nube en la dirección y sentido indicados por el vector (vx,vy).
15. Dado un valor de TipoCuadrante devuelva un Punto con la suma de todas las coordenadas de los puntos de ese cuadrante.
16. Devuelva un SortedMap que asocie a cada TipoCuadrante el Punto cuyas coordenadas sean la suma de todas las coordenadas de los puntos de ese cuadrante.
17. Devuelva un SortedMap que asocie a cada TipoCuadrante el Punto centro de gravedad de los puntos de ese cuadrante.
18. Proporcione a la clase NubePuntos un constructor público tal que reciba el nombre de un fichero conteniendo en cada línea dos valores reales separados por una coma con las coordenadas de los puntos y un constructor privado que reciba una List<Punto> para usarlo en el ejercicio 10.

Implemente todos los métodos anteriores en dos clases para NubePuntos, una denominada NubePuntosImpl una usando los esquemas iterativos habituales y otra NubePuntosImpl2 usando los métodos de stream de Java 8. Compare los resultados.

11.3 Soluciones

```
public enum TipoCuadrante {  
    PRIMER, SEGUNDO, TERCER, CUARTO, EJES  
}
```

```
public enum TipoDiana {  
    INTERIOR, MEDIO, EXTERIOR  
}
```

```
public interface Punto extends Comparable<Punto>{  
    Double getX();  
    Double getY();  
    void setX(Double nx);  
    void setY(Double ny);  
    Double getDistanciaOtroPunto(Punto q);  
    Double getDistanciaAlOrigen();  
    TipoCuadrante getCuadrante();  
    TipoDiana getPosicionDiana();  
    void moverDireccion(Double nx, Double ny);  
}
```

```
public class PuntoImpl implements Punto {  
    Double x;  
    Double y;  
    static final Punto ORIGEN = new PuntoImpl(0.,0.);  
  
    public PuntoImpl(Double nx, Double ny){  
        x=nx;
```

```

        y=ny;
    }
    public PuntoImpl(String s){
        List<String> lisat = Arrays.asList(s.split(","));
        x=new Double(lisat.get(0));
        y=new Double(lisat.get(1));
    }
    public Double getX() {
        return x;
    }
    public Double getY() {
        return y;
    }
    public void setX(Double nx) {
        x=nx;
    }
    public void setY(Double ny) {
        y=ny;
    }
    public String toString(){
        String cadena;
        cadena="("+getX()+","+getY()+")";
        return cadena;
    }
    public Double getDistanciaOtroPunto(Punto q){
        Double dx = this.getX()-q.getX();
        Double dy = this.getY()-q.getY();
        return Math.sqrt(dx*dx+dy*dy);
    }

    public Double getDistanciaAlOrigen(){
        return this.getDistanciaOtroPunto(ORIGEN);
    }

    public boolean equals(Object obj){
        boolean res=false;

        if (obj instanceof Punto){
            Punto p = (Punto) obj;
            res= this.getX().equals(p.getX()) &&
                this.getY().equals(p.getY());
        }
        return res;
    }
    public TipoCuadrante getCuadrante(){
        TipoCuadrante t=TipoCuadrante.EJES;
        if (this.getX()>0. && this.getY()>0.){
            t=TipoCuadrante.PRIMER;
        }else if (this.getX()<0. && this.getY()>0.){
            t=TipoCuadrante.SEGUNDO;
        }else if (this.getX()<0. && this.getY()<0.){
            t=TipoCuadrante.TERCER;
        }else if (this.getX()>0. && this.getY()<0.){
            t=TipoCuadrante.CUARTO;
        }
        return t;
    }
    public TipoDiana getPosicionDiana(){
        TipoDiana t=TipoDiana.EXTERIOR;

        if (this.getDistanciaAlOrigen()<4.0){
            t=TipoDiana.INTERIOR;
        }
        else if (this.getDistanciaAlOrigen()<8.0){
            t=TipoDiana.MEDIO;
        }
    }

```

```

    }
    return t;
}
public int compareTo(Punto p){
    int res;
    res=this.getX().compareTo(p.getX());
    if (res==0){
        res=this.getY().compareTo(p.getY());
    }
    return res;
}
public void moverDireccion(Double nx, Double ny){
    this.setX(this.getX()+nx);
    this.setY(this.getY()+ny);
}
}

```

1. Un método tal que dado un valor real nx devuelva el número de puntos que tienen abscisa mayor de nx.

```

// Esquema de filtro+contador. Normalmente los contadores se definirían Integer pero por
// compatibilidad con el método count de Java 8 devolvemos un Long
public Long getNumPuntosAbsMayor(Double nx) {
    Long cont=0L; // La L indica que es un Long
    for (Punto p:nube){
        if (p.getX().compareTo(nx)>0){
            cont++;
        }
    }
    return cont;
}

```

```

// Java 8
public Long getNumPuntosAbsMayor(Double nx) {
    return nube.stream().filter(x->x.getX().compareTo(nx)>0).count();
}

```

2. Un método tal que dado un valor de TipoCuadrante devuelva el número de puntos en ese cuadrante, no olvide los puntos sobre los ejes. (esquema clásico de filtro+contador)

```

public Long getNumPuntosCuadrante(TipoCuadrante t){
    Long cont=0L; // La L indica que es un Long
    for(Punto p: nube){
        if (p.getCuadrante().equals(t)){
            cont++;
        }
    }
    return cont;
}

```

```

// Java 8
public Long getNumPuntosCuadrante(TipoCuadrante t){
    return nube.stream().filter(p->p.getCuadrante().equals(t)).count();
}

```

3. Un método tal que dado un valor real d devuelva el número de puntos cuya distancia al origen es menor que d.

```

// igual que los anteriores
public Long getNumPuntosDistOrigenMenor(Double d) {
    Long cont= 0L;
    for (Punto p:nube){
        if (p.getDistanciaAlOrigen().compareTo(d)<0){

```

```
        cont++;  
    }  
    return cont;  
}
```

```
// Java 8  
public Long getNumPuntosDistOrigenMenor(Double d) {  
    return nube.stream().  
        filter(x->x.getDistanciaAlOrigen().compareTo(d)<0).count();  
}
```

4. Devuelva un Punto que sea el centro de gravedad de la nube. Esto es, el punto cuyas coordenadas son la media de las coordenadas de los puntos.

```
// Esquema de suma con media aritmética al final
public Punto getCentroGravedad() {
    Double sumx=0.,sumy=0.;
    for(Punto p:nube){
        sumx = sumx + p.getX();
        sumy = sumy + p.getY();
    }
    return new PuntoImpl(sumx/nube.size(),sumy/nube.size());
}
```

```
// En Java 8 podemos trabajar con la media directamente
public Punto getCentroGravedad() {
    Double avgx = nube.stream().
        collect(Collectors.averagingDouble(Punto::getX));
    Double avgy = nube.stream().
        collect(Collectors.averagingDouble(Punto::getY));
    return new PuntoImpl(avgx,avgy);
}
```

5. Devuelva si existe algún punto sobre el eje de abscisas.

```
// esquema existe
public Boolean existePuntoEjeAbscisas() {
    Boolean res=false;

    for(Punto p:nube){
        if (p.getY().equals(0.)){
            res=true;
        }
    }
    return res;
}
```

```
// En Java 8
public Boolean existePuntoEjeAbscisas() {
    return nube.stream().anyMatch(x->x.getY().equals(0.));
}
```

6. Dado un Punto p y un radio r devuelva si existe algún punto dentro del círculo de centro p y radio r. (Esquema existe)

```
// esquema existe
public Boolean existePuntoDentroCirculo(Punto c, Double r) {
    Boolean res=false;
    for(Punto p:nube){
        if (p.getDistanciaOtroPunto(c).compareTo(r)<0){
            res=true;
        }
    }
    return res;
}
```

```
// En Java 8
public Boolean existePuntoDentroCirculo(Punto c, Double r) {
    return nube.stream().
        anyMatch(x->x.getDistanciaOtroPunto(c).compareTo(r)<0);
}
```

7. Dado un valor real d, devuelve si es cierto que todos los puntos están a una distancia al origen menor que d.

```
// esquema para todo
public Boolean estanTodosADistMenor(Double d) {
    Boolean res=true;
    for(Punto p:nube){
        if (p.getDistanciaAlOrigen().compareTo(d)>0){
            res=false;
        }
    }
    return res;
}
```

```
// En Java 8
public Boolean estanTodosADistMenor(Double d) {
    return nube.stream().
        allMatch(x->x.getDistanciaAlOrigen().compareTo(d)<0);
}
```

8. Devuelve el Punto más lejano al origen de coordenadas.

```
//Máximo usando un comparator y el método max de la clase Collections. En Java 7 el
// Comparator es una clase que se debe definir aparte
public class ComparadorDistanciaOrigen implements Comparator<Punto> {

    public int compare(Punto p, Punto q){
        return p.getDistanciaAlOrigen().compareTo(q.getDistanciaAlOrigen());
    }

}

public Punto getMasLejanoOrigen() {
    return Collections.max(nube, new ComparadorDistanciaOrigen());
}
```

```
// En Java 8 no hace falta definir la clase Comparator sino que usamos la propiedad
// distanciaAlOrigen de Punto junto con el método comparing. El get es necesario por si la
// colección está vacía.
public Punto getMasLejanoOrigen() {
    return nube.stream().
        max(Comparator.comparing(Punto::getDistanciaAlOrigen)).get();
}
```

9. Dado un Punto p, devuelve el punto de la nube más cercano.

```
// Es un problema de mínimo que tiene la dificultad de que el Comparator debe tener un
// atributo (base) que es el punto al que le calculamos el vecino y que debe ser argumento
// en el constructor
public class ComparadorDistanciaPunto implements Comparator<Punto> {
    Punto base;

    public ComparadorDistanciaPunto(Punto p){
        base=p;
    }
    public int compare(Punto p1, Punto p2){
        return p1.getDistanciaOtroPunto(base).compareTo(p2.getDistanciaOtroPunto(base));
    }
}

public Punto getPuntoMasCercano(Punto p) {
    return Collections.min(nube, new ComparadorDistanciaPunto(p));
}
```

```
// Esto es un mínimo con Comparator aunque en Java 8 ahora no nos basta con invocar a un
// método get, sino que hay que definir el Comparator, aunque no es necesaria una clase y
// se puede hacer en el mismo código de NubeImpl mediante una lambda expresión.
public Comparator<Punto> comparadorDistanciaPunto(Punto base){
    return (x,y)->
        x.getDistanciaOtroPunto(base).compareTo(y.getDistanciaOtroPunto(base));
}

public Punto getPuntoMasCercano(Punto p) {
    return nube.stream().min(new ComparadorDistanciaPunto(p)).get();
}
```

10. Dado un valor real d, devuelva una NubePuntos con aquellos que estén a una distancia menor que d del origen.

```
// Este es un esquema de filtro, como queremos encapsular la colección de Punto devolvemos
// un objeto NubePuntos y no un List<Punto>. Para ello nos hace falta un constructor de
// NubePunto a partir de un List<Punto>. Este constructor será private y sólo se usará en
// este método. El constructor público será leyendo desde un fichero. De esta manera el
// atributo de tipo List<Punto> queda oculto.

public NubePuntos getPuntosMenorDistanciaOrigen (Double d){
    List<Punto> fil=new ArrayList<Punto>();
    for (Punto p:nube){
        if (p.getDistanciaAlOrigen().compareTo(d)<0){
            fil.add(p);
        }
    }
    return new NubePuntosImpl(fil);
}
```

```
// En Java 8 el filtro se consigue con el método filter para seleccionar y el método
// collect para devolver una Colección de tipo List en este caso.
public NubePuntos getPuntosMenorDistanciaOrigen (Double d){
    List<Punto> fil = nube.stream().
        filter(x->x.getDistanciaAlOrigen().compareTo(d)<0).
        collect(Collectors.toList());
    return new NubePuntosImpl2(fil);
}
```

11. Devuelva un Map que asocie a cada TipoCuadrante un SortedSet con los puntos de ese cuadrante ordenados por distancia al origen.

```
// Esquema de construcción de un Map donde el tipo value es mutable
public Map<TipoCuadrante, SortedSet<Punto>> getPuntosPorCuadrante(){
    Map<TipoCuadrante, SortedSet<Punto>> res = new
        HashMap<TipoCuadrante, SortedSet<Punto>>();

    for (Punto p: nube){
        TipoCuadrante clave = p.getCuadrante();
        if(res.containsKey(clave)){
            res.get(clave).add(p);
        }
        else{
            SortedSet<Punto> sp = new TreeSet<Punto>(
                new ComparadorDistanciaOrigen());
            sp.add(p);
            res.put(clave, sp);
        }
    }
    return res;
}
```

```

// En Java 8 un Map se crea mediante el método groupingBy de Collectors al que le
// proporcionamos la propiedad por la que se forman las claves (getCuadrante) y que el
// conjunto de valores está formado por una Collection mediante un conjunto ordenado por
// un Comparator
public Map<TipoCuadrante, SortedSet<Punto>> getPuntosPorCuadrante(){
    return nube.stream().
        collect(Collectors.groupingBy(Punto::getCuadrante,
            Collectors.toCollection(()->new TreeSet<Punto>(
                new ComparadorDistanciaOrigen()))));
}

```

12. Devuelva un Map que asocie a cada TipoCuadrante el número de puntos que hay de ese tipo.

```

// esquema de construcción de un Map donde los valores son inmutables. El tipo contador es
// Long para hacerlo compatible con Java 8
public SortedMap<TipoCuadrante, Long> getNumPuntosPorCuadrante(){
    SortedMap<TipoCuadrante, Long> res = new TreeMap<TipoCuadrante, Long>();
    for (Punto p: nube){
        TipoCuadrante clave = p.getCuadrante();
        if(res.containsKey(clave)){
            Long val=res.get(clave);
            val++;
            res.put(clave, val);
        }
        else{
            res.put(clave, 1L);
        }
    }
    return res;
}

```

```

// En Java 8 de nuevo invocamos a groupingBy, el supplier del TreeMap nos hace devolver un
// SortedMap y en el conjunto de valores se calcula como un contador mediante el método
// counting
public SortedMap<TipoCuadrante, Long> getNumPuntosPorCuadrante(){
    return nube.stream().
        collect(Collectors.groupingBy(
            Punto::getCuadrante,
            TreeMap::new,
            Collectors.counting()));
}

```

13. Devuelva un SortedMap que asocie a cada TipoDiana un Set con los puntos que le corresponden.

```

// Este es un esquema similar a ejercicio 11, cambia que el Map es ordenado y los
// elementos del conjunto Values es un Set en vez de un SortedSet, pero eso no cambia nada
// del método salvo la invocación a los constructores
public SortedMap<TipoDiana, Set<Punto>> getPuntosPorDiana(){
    SortedMap<TipoDiana, Set<Punto>> res = new TreeMap<TipoDiana, Set<Punto>>();
    for (Punto p: nube){
        TipoDiana clave = p.getPosicionDiana();
        if(res.containsKey(clave)){
            res.get(clave).add(p);
        }
        else{
            Set<Punto> sp = new HashSet<Punto>();
            sp.add(p);
            res.put(clave, sp);
        }
    }
    return res;
}

```



```
// En Java 8 cambia respecto de la solución del 11 que hay que proporcionar el supplier
// del TreeMap (Como ya hicimos en el 12) y que al ser un Set es másfácil, pues basta con
// invocar al método toSet de Collectors (y no al método general toCollection de la
// solución del ejercicio 11.
public SortedMap<TipoDiana, Set<Punto>> getPuntosPorDiana(){
    return nube.stream().
        collect(Collectors.groupingBy(
            Punto::getPosicionDiana,
            TreeMap::new,
            Collectors.toSet()));
}
```

14. Dados dos valores vx e vy desplace toda la nube en la dirección y sentido indicados por el vector (vx,vy)

```
public void moverDireccion(Double vx, Double vy){
    for(Punto p:nube){
        p.moverDireccion(vx, vy);
    }
}
```

```
public void moverDireccion(Double vx, Double vy){
    nube.stream().forEach(x->x.moverDireccion(vx,vy));
}
```

15. Dado un valor de TipoCuadrante devuelva un Punto con la suma de todas las coordenadas de los puntos de ese cuadrante.

```
// este problema es prácticamente igual al ya se ha resuelto en el ejercicio 4 y en Java 7
// no aporta ninguna novedad
public Punto getPuntoSuma(TipoCuadrante t){
    Double sx=0.,sy=0.;
    for(Punto p:nube){
        if (p.getCuadrante().equals(t)){
            sx=sx+p.getX();
            sy=sy+p.getY();
        }
    }
    return new PuntoImpl(sx,sy);
}
```

```
// Sin embargo en Java 8 es un interesante ejercicio de aplicación del método reduce con
// un operador binario (caso particular de BiFunction). Si definimos el BinaryOperator que
// devuelve el Punto suma de dos, entonces proporcionamos al método reduce el elemento
// neutro y el operador binario para que devuelva la operación acumulada sobre todos los
// elementos del stream que pasan el filtro.
BinaryOperator<Punto> sumaCoord = (x,y)-> {
    Double nx = x.getX() + y.getX();
    Double ny = x.getY() + y.getY();
    return new PuntoImpl(nx,ny);
};
public Punto getPuntoSuma(TipoCuadrante t){
    return nube.stream().
        filter(x->x.getCuadrante().equals(t)).
        reduce(new PuntoImpl(0.,0.), sumaCoord);
}
```

```
// De hecho de haber definido anteriormente el operador binario sumaCoord el ejercicio 4
// podría haberse resuelto de esta manera:
public Punto getCentroGravedad() {
    Punto m = nube.stream().reduce(new PuntoImpl(0.,0.), sumaCoord);
    m.setX(m.getX()/nube.size());
    m.setY(m.getY()/nube.size());
    return m;
}
```

16. Devuelva un SortedMap que asocie a cada TipoCuadrante el Punto cuyas coordenadas sean la suma de todas las coordenadas de los puntos de ese cuadrante.

```
// Este problema es una generalización del anterior para conseguir relacionar cada
// Cuadrante con su centro de masa resultado de sumar todos los puntos en uno solo.
// Nótese que la media (centro de gravedad) no sería posible de calcular hasta el final
// y que deberíamos invocar al método que nos devuelve el número de puntos en cada
// cuadrante para hallar la media. Obsérvese que el tipo de los objetos de valores es
// mutable, por lo que es necesario crear uno nuevo cada vez, ya que modificar el Punto
// de valores conllevaría indeseados efectos laterales
public SortedMap<TipoCuadrante, Punto> getPuntoSumaPorCuadrante(){
    SortedMap<TipoCuadrante, Punto> res = new TreeMap<TipoCuadrante, Punto>();

    for (Punto p: nube){
        TipoCuadrante clave = p.getCuadrante();
        if(res.containsKey(clave)){
            Punto val = res.get(clave);
            val = new PuntoImpl(val.getX()+p.getX(),val.getY()+p.getY());
            res.put(clave, val);
        }
        else{
            res.put(clave, p);
        }
    }
    return res;
}
```

```
// En Java 8 es una solución similar a los problemas 12 y 13, pero usando el método
// reducing de Collectors para aprovechar el operador binario sumaCoord definido en la
// solución del ejercicio 15
public SortedMap<TipoCuadrante, Punto> getPuntoSumaPorCuadrante(){
    return nube.stream().
        collect(Collectors.groupingBy(
            Punto::getCuadrante,
            TreeMap::new,
            Collectors.reducing(new PuntoImpl(0.,0.),sumaCoord)));
}
```

17. Devuelva un SortedMap que asocie a cada TipoCuadrante el Punto centro de gravedad de los puntos de ese cuadrante.

```
// Reutilizando los Map resultado de la suma de puntos y el número de puntos por
// Cuadrante basta con cambiar el punto suma por el punto medio
public SortedMap<TipoCuadrante, Punto> getCentroGravedadPorCuadrante(){
    SortedMap<TipoCuadrante, Punto> m1 = getPuntoSumaPorCuadrante();
    SortedMap<TipoCuadrante, Long> m2 = getNumPuntosPorCuadrante();

    for (TipoCuadrante k:m1.keySet()){
        Punto sum = m1.get(k);
        Long num = m2.get(k);
        sum.setX(sum.getX()/num);
        sum.setY(sum.getY()/num);
    }
    return m1;
}
```

```
// En Java 8 el for para recorrer los cuadrantes de las claves del Map se implementa
// mediante un método forEach
public SortedMap<TipoCuadrante, Punto> getCentroGravedadPorCuadrante(){
    SortedMap<TipoCuadrante, Punto> m1 = getPuntoSumaPorCuadrante();
    SortedMap<TipoCuadrante, Long> m2 = getNumPuntosPorCuadrante();

    m1.keySet().forEach(k->m1.get(k).setX(m1.get(k).getX()/m2.get(k)));
    m1.keySet().forEach(k->m1.get(k).setY(m1.get(k).getY()/m2.get(k)));
    return m1;
}
```

18. Proporcione a la clase NubePuntos un constructor público tal que reciba el nombre de un fichero conteniendo en cada línea dos valores reales separados por una coma con las coordenadas de los puntos y un constructor privado que reciba una List<Punto> para usarlo en el ejercicio 10.

```
// En Java 7 mantenemos la lectura tradicional mediante el método estático
// leeFicheroPorLinea en la clase Util que devuelva el fichero leído por líneas,
// siendo cada línea las coordenadas de un Punto
private NubePuntosImpl(List<Punto> lis){
    nube = lis;
}
public NubePuntosImpl(String nomFile){
    List<String> ls = Util.LeeFicheroPorLinea(nomFile);
    nube = new ArrayList<Punto>();
    for (String linea:ls){
        Punto p = new PuntoImpl(linea);
        nube.add(p);
    }
}
```

```
// en la clase Util
public static List<String> LeeFicheroPorLinea(String fileName) {
    List<String> listaleida = new LinkedList<String>();
    try {
        Scanner sc = new Scanner(new File(fileName));
        while (sc.hasNextLine()){
            listaleida.add(sc.nextLine());
        }
        sc.close();
    } catch (FileNotFoundException e){
        System.out.println("Fichero no encontrado"+fileName);
    }
    return listaleida;
}
```

```
// En Java 8 la lectura de un fichero por líneas se hace mediante el método lines de
// la clase Files. Convertir el stream de String a List de Punto es trivial con los
// métodos map (y el supplier del constructor de Punto a partir de un String) y el
// método collect.
public NubePuntosImpl2(String nomFile){
    try {
        nube = Files.lines(Paths.get(nomFile)).
            map(x->new PuntoImpl(x)).
            collect(Collectors.toList());
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```